

Name : Suraj Haridas Jagtap

College Name : Government Polytechnic Achalpur

Domain : AI/ML

Q1. (Data Loading & Preprocessing)

```
import pandas as pd

df=pd.read_csv("heart - heart.csv")

df
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope
0	40	M	ATA	140	289	0	Normal	172	N	0.0	
1	49	F	NAP	160	180	0	Normal	156	N	1.0	
2	37	M	ATA	130	283	0	ST	98	N	0.0	
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	
4	54	M	NAP	150	195	0	Normal	122	N	0.0	
...	...	...	...	...	...	...	...	...	...	...	...
913	45	M	TA	110	264	0	Normal	132	N	1.2	
914	68	M	ASY	144	193	1	Normal	141	N	3.4	
915	57	M	ASY	130	131	0	Normal	115	Y	1.2	
916	57	F	ATA	130	236	0	LVH	174	N	0.0	
917	38	M	NAP	138	175	0	Normal	173	N	0.0	

918 rows × 12 columns

```
y.head()
```

```
0    0
1    1
2    0
3    1
4    0
Name: HeartDisease, dtype: int64
```

Q2. (Train-Test Split)

```
from sklearn.model_selection import train_test_split
```

Python

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
```

Python

```
print("X_train:", X_train.shape, "X_test:", X_test.shape, "y_train:", y_train.shape, "y_test:", y_test.shape)
```

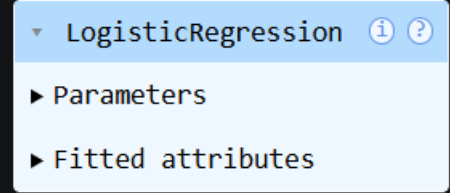
Python

```
X_train: (734, 15) X_test: (184, 15) y_train: (734,) y_test: (184,)
```

Q3. (Building Logistic Regression Model)

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
```

```
n_iter_1 = _check_optimize_result(
```



The image shows a Jupyter Notebook interface. At the top, a code cell contains the following Python code: `from sklearn.linear_model import LogisticRegression`, `model = LogisticRegression()`, and `model.fit(X_train, y_train)`. Below the code cell, the variable explorer is open, showing a variable named `LogisticRegression`. The variable explorer has a search bar and a scroll bar. The `LogisticRegression` variable is expanded, showing two sections: `Parameters` and `Fitted attributes`. The `Parameters` section is currently selected and expanded, showing a list of parameters: `C`, `class_weight`, `decision_function_shape`, `dual`, `fit_intercept`, `l1_penalty`, `l2_penalty`, `max_iter`, `max_iter_burnin`, `max_iter_warning`, `multi_class`, `n_jobs`, `penalty`, `random_state`, `solver`, `tol`, `verbose`, and `warm_start`. The `Fitted attributes` section is also visible but not expanded.

Q4. (Making Predictions)

```
16] y_pred = model.predict(X_test)

> ~ print("Actual:", y_test[:10].values,"Predicted:", y_pred[:10])

17]

... Actual: [0 1 1 1 0 1 1 0 1 1] Predicted: [0 0 1 1 0 1 1 0 1 1]
```

Q5. (Confusion Matrix)

```
from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
cm
```

```
TN, FP, FN, TP = cm.ravel()
```

```
print("True Negative:", TN)
```

```
print("False Positive:", FP)
```

```
print("False Negative:", FN)
```

```
print("True Positive:", TP)
```

```
18]
```

```
True Negative: 66  
False Positive: 11  
False Negative: 16  
True Positive: 91
```

Q6. (Model Evaluation Metrics)

```
from sklearn.metrics import classification_report

report = classification_report(y_test, y_pred)
report
```

```

              precision    recall  f1-score   support

     0       0.80      0.86      0.83        77
     1       0.89      0.85      0.87       107

 accuracy          0.85
 macro avg       0.85
weighted avg       0.86
```

Q7. (Saving the Model)

```
> ~  
import joblib  
  
joblib.dump(model, "heart_model.pkl")  
  
joblib.dump(scaler, "scaler.pkl")  
  
joblib.dump(X.columns.tolist(), "columns.pkl")  
[ ]  
... ['columns.pkl']
```

Q8. (Loading & Testing Saved Model)

```
> model = joblib.load("heart_model.pkl")
columns = joblib.load("columns.pkl")

sample = {
    "Age": 52,
    "Sex": "M",
    "ChestPainType": "ATA",
    "Cholesterol": 200,
    "RestingBP": 120,
    "FastingBS": 0,
    "RestingECG": "Normal",
    "MaxHR": 150,
    "ExerciseAngina": "N",
    "Oldpeak": 1.2,
    "ST_Slope": "Up"
}

sample_df = pd.DataFrame([sample])

sample_df = pd.get_dummies(sample_df)
sample_df = sample_df.reindex(columns=columns, fill_value=0)

sample_df = scaler.transform(sample_df)

prediction = model.predict(sample_df)[0]

print("Prediction:", "Heart Disease" if prediction == 1 else "No Heart Disease")
```

25]

```
.. Prediction: No Heart Disease
```

Q9. (Streamlit Input Interface)

```
> ~
model = joblib.load("heart_model.pkl")
columns = joblib.load("columns.pkl")

sample = {
    "Age": 52,
    "Sex": "M",
    "ChestPainType": "ATA",
    "Cholesterol": 200,
    "RestingBP": 120,
    "FastingBS": 0,
    "RestingECG": "Normal",
    "MaxHR": 150,
    "ExerciseAngina": "N",
    "Oldpeak": 1.2,
    "ST_Slope": "Up"
}

sample_df = pd.DataFrame([sample])

sample_df = pd.get_dummies(sample_df)
sample_df = sample_df.reindex(columns=columns, fill_value=0)

sample_df = scaler.transform(sample_df)

prediction = model.predict(sample_df)[0]

print("Prediction:", "Heart Disease" if prediction == 1 else "No Heart Disease")

25]
.. Prediction: No Heart Disease
```

Heart Disease Prediction App

Age

50

- +

Sex

M

▼

Chest Pain Type

ATA

▼

Cholesterol

200

- +

RestingBP

120

- +

FastingBS

0

▼

RestingECG

Normal

▼

MaxHR

150

- +

ExerciseAngina

Y

▼

Oldpeak

1.00

- +

ST_Slope

Up

▼

Predict

Q10. (Complete Streamlit Web App)

```
app.py / ...
1  import streamlit as st
2  import pandas as pd
3  import joblib
4
5  model = joblib.load("heart_model.pkl")
6  columns = joblib.load("columns.pkl")
7
8  st.title("Heart Disease Prediction App")
9
10 age = st.number_input("Age", min_value=1, max_value=120, value=50)
11 sex = st.selectbox("Sex", ["M", "F"])
12 chest_pain = st.selectbox("Chest Pain Type", ["ATA", "NAP", "ASY", "TA"])
13 cholesterol = st.number_input("Cholesterol", min_value=0, max_value=600, value=200)
14 resting_bp = st.number_input("RestingBP", min_value=0, max_value=250, value=120)
15 fasting_bs = st.selectbox("FastingBS", [0, 1])
16 resting_ecg = st.selectbox("RestingECG", ["Normal", "ST", "LVH"])
17 max_hr = st.number_input("MaxHR", min_value=60, max_value=220, value=150)
18 exercise_angina = st.selectbox("ExerciseAngina", ["Y", "N"])
19 oldpeak = st.number_input("Oldpeak", min_value=0.0, max_value=10.0, value=1.0)
20 st_slope = st.selectbox("ST_Slope", ["Up", "Flat", "Down"])
21
22 if st.button("Predict"):
23
24     sample = {
25         "Age": age,
26         "Sex": sex,
27         "ChestPainType": chest_pain,
28         "Cholesterol": cholesterol,
29         "RestingBP": resting_bp,
30         "FastingBS": fasting_bs,
31         "RestingECG": resting_ecg,
32         "MaxHR": max_hr,
33         "ExerciseAngina": exercise_angina,
34         "oldpeak": oldpeak,
35         "ST_Slope": st_slope
36     }
37
38     sample_df = pd.DataFrame([sample])
39     sample_df = pd.get_dummies(sample_df)
40     sample_df = sample_df.reindex(columns=columns, fill_value=0)
41
42     prediction = model.predict(sample_df)[0]
43
44     if prediction == 1:
45         st.error("Heart Disease: Yes")
46     else:
47         st.success("Heart Disease: No")
48
```

Heart Disease Prediction App

Age

50

-

+

Sex

M

▼

Chest Pain Type

ATA

▼

Cholesterol

200

-

+

RestingBP

120

-

+

FastingBS

0

▼

RestingECG

Normal

▼

MaxHR

150

-

+

ExerciseAngina

Y

▼

Oldpeak

1.00

-

+

ST_Slope

Up

▼

Predict

Heart Disease: No